Ex 2.2.15: lines cut short by the training window

> The anchor pulls on whole lines, but each training window cuts off the lines at its edges. On a cut line, the anchor asks the visible part to carry the whole label, even when that part cannot see the op word.
>
> We retrain the anchored model under a few policies for which cut lines to pull, and track each one over training. We do this on the current grammar, before the in-context grammar makes the problem larger.
>
> The cut lines carry the lean, and skipping them removes it. The pilot starts from the policy that pulls whole lines only.

This is a scouting run of 55 fresh training runs, with a few predictions and one rule that proposes which crop policy the in-context grammar pilot starts from.

Cut lines are a small share of any one batch, but the anchor meets them at every step, so the question is what they add up to over training. Each run records the lean and the trailing-fragment lean at every trajectory point (every 50 training steps) and at the end.

Findings

- The first operand lean is due to cut lines (H1) — **held**: skipping the cut lines removes the lean, and pulling only the cut lines leaves more of it than pulling every line.
- The anchor and the task hold under every policy (H2) — **did not hold**: the anchor and the task hold under the policies the rule weighs; `cut-only` loses some margin, and both halved-window arms cost task accuracy on `difference`.
- Trailing fragments without their op word (H3) — **held**: fragments cut off before their op word lean far more than a first operand does, and skipping them removes that lean.
- Halved windows, more cut lines (H4) — **partly held**: halving the window shrank the gap between `all` and `whole` where we expected it to grow, and `whole-short` still removes the lean.
- Where the rest of the lean comes from (H5) — **held**: `whole` leaves no lean for the tied readout or the newline mask to remove.

The rule for the pilot: the rule chose `half`, but the pilot starts from **`whole`**, which skips every cut line.

How to read this draft

The policies, the five predictions, and the rule were fixed before any run, at commit `5334cb9`. Everything after that commit is either results filled into their sections or exploratory work, marked as post hoc.

Each result section opens with what we expect, then what we saw, then what we make of it.

Why this experiment

The training corpus is one long tape of six-token lines. Each training step cuts a batch of 64 windows of 64 tokens from it, each at a random place. Most lines in a window are whole, but its two edges usually slice through a line.

The anchor does not know about the edges. It asks each labelled line to align with the axis at whichever visible position aligns most easily. So when a line is cut, the visible part alone has to carry the whole label.

In these runs the anchor targets the op `difference`, as in the primary arm of ex-2.2.14. If a `difference` line shows only its first operand, the anchor asks a color token to say "this line is `difference`" before the op word has appeared. That token cannot know, so the only way the model can satisfy the anchor there is to lean every first operand a little toward the axis.

That is what ex-2.2.14 saw, after the fact. At the last block, the first operand leaned toward e_1 at 0.19 on the primary arm, against -0.01 on the control, and the arm that pulls only the op word had no lean. The op1-lean reanalysis found the same route on the *red* runs, and proposed skipping the pull on cut lines as a test.

Before fixing the predictions, we ran that test as a smoke test on *red*: two seeds each of the conditions `all`, `whole`, and `cut-only`. `whole` took the lean from 0.23 to 0.17, and `cut-only` kept 0.08, against about zero for the control. So on *red*, cut lines carry about a third of the lean and whole lines the rest. The validation loss was the same under every policy, to three decimals.

Where the rest comes from is a question of its own. So two more arms keep `whole` and change the model. One stops attention at each newline; the other ties the readout to the embedding table (the output layer reuses the input embedding weights).

Red is a weaker test than the op, though. On a *red* line the first operand can be red itself, which is evidence the anchor can pull on; on a `difference` line the first operand says nothing about the op.

At 64 tokens, 15% of line visits are cut short. On 7.4% the op word is out of sight, and on 1.5% only the first operand is visible. The shares are small, but the term is normalized per labelled line, so each of those visits gets the pull of a full line.

This matters more after the D2.2 pivot. There, a line is a context of solved examples about 20 tokens long, and the op is inferred from the examples, so a window that cuts the start of a context can remove the examples that name the op. With fewer lines per window, the share of cut visits roughly doubles, and a cut context has lost evidence, not just one token. The current grammar is the cheap place to see what each policy does, with a clear sign (the lean) to watch.

Glossary

Cut line

A line visit that the training window shows only part of. A line cut at its start has lost its first tokens, so its states differ from those of the whole line. A line cut at its end shows a prefix. Under causal attention (each position sees only earlier positions), those prefix states are the same as in the whole line.

Crop policy

Which labelled lines the anchor pulls, given what the window shows of them. The label draws are the same under every policy; only the pull changes.

Pull kept

The share of the current total pull a policy keeps, averaged over every way a window shows a line. It is computed from the sampler, not measured.

The lean

The mean cosine with e_1 at the first operand, over every op's probe lines, at the last block. The first operand comes before the op word, so any lean there is shared by every line, whatever its op.

Trailing fragment

A probe line shown from its second operand, `=`, answer, or newline on, as a sequence of its own: what a window leaves of a line when it cuts off the op word.

Op margin

As in ex-2.2.14: how much closer the `difference` lines sit to e_1 than the pool of all eleven ops, at the role where the gap is largest, averaged over slices. It checks that the anchor landed.

Conditions

condition	which labelled lines the anchor pulls	model	window	pull kept	seeds
`all`	every labelled line, whatever is visible (the current behaviour)	untied readout	64	100%	5
`whole`	only lines wholly inside the window	untied readout	64	85%	5
`half`	only lines with more than half their tokens inside the window	untied readout	64	91%	5
`scaled`	every labelled line, its pull scaled by the share of it that is visible	untied readout	64	93%	5
`knowable`	only positions at or after a visible op word	untied readout	64	93%	5
`cut-only`	only lines the window cuts short	untied readout	64	15%	5
`all-short`	every labelled line, whatever is visible (the current behaviour)	untied readout	32	100%	5
`whole-short`	only lines wholly inside the window	untied readout	32	73%	5
`whole-mask`	only lines wholly inside the window	attention stops at each newline	64	85%	5
`whole-tied`	only lines wholly inside the window	tied readout	64	85%	5
`all-tied`	every labelled line, whatever is visible (the current behaviour)	tied readout	64	100%	5
`control`	none: the un-anchored control from ex-2.2.11, served from the store	untied readout	64	–	5

Every arm is the primary from ex-2.2.14: `difference` on e_1, labelled at a rate of 0.02 per line, the pull over the whole line, and the handover recipe. Only the crop policy changes, plus the window for the short pair and the model for the last three.

Same batches, same labels. A policy is a weight on the pull of each labelled line, applied after the labels are drawn. So at one seed every 64-token arm trains on the same windows with the same labels, and differences between arms at a seed come from the policy. The model arms, `whole-mask`, `whole-tied`, and `all-tied`, share those windows and labels too, so each pairs with `whole` or `all` at a seed. The short-window arms draw differently and are compared with each other.

A policy only takes pull away. A line that a policy keeps gets the same pull it had under `all`, because the term still divides by the number of labelled lines with anything visible. We could instead divide by the number of lines the policy keeps, but then every kept pull would grow stronger as the policy drops more, which is the side effect ex-2.1.7 warned about.

The cost of our choice is that the policies differ a little in total pull, so the table gives the share each keeps. `cut-only` keeps only 15%, and that is intended: `whole` and `cut-only` split the pull of `all` in two, so if the lean follows `cut-only`, it follows the cut lines and not the total.

The policies. `whole`, `half`, and `scaled` need nothing but the window, so each would carry over to any grammar. They differ on spans longer than the window, as a natural-language document often is: `whole` never pulls one, `half` stops at twice the window, and `scaled` pulls every span by the share in view.

`knowable` needs to know where the evidence is. Here, that is the op word; in the in-context grammar, it would be the posterior over ops given the tokens so far (label variant (c) in the pivot). It also drops the first operand from the pull on whole lines, as the note in ex-2.2.14 suggested, and is a reference for what knowing the evidence buys.

The halved windows. At 32 tokens, 27% of line visits are cut, about the share the in-context grammar would have. The batch doubles to 128 windows, so a step sees the same number of tokens and the runs take the same steps. `all-short` against `whole-short` asks whether the effect grows with the share of cut lines, and whether `whole` still removes it.

The model arms. `whole-mask` and `whole-tied` each keep `whole` and change one thing about the model, to test a route for any lean that `whole` leaves on whole lines. `whole-mask` stops attention at each newline, so a position sees only its own line; on a whole line the first operand then sees nothing but itself. The pivot names this mask as optional for the in-context grammar. `whole-tied` ties the readout to the embedding table, as `handover-tied` did in ex-2.2.9. The untied readout arrived with the handover recipe, and the op1-lean reanalysis found the lean on *red* running through it. `all-tied` pairs with `whole-tied`, so the share of the lean that cut lines carry can be measured under the tied readout as well.

The first operand lean is due to cut lines (H1)

✓ Pass

What we expect. `all` reproduces the lean from ex-2.2.14. `whole` removes it: its seed-mean lean is within 0.05 of the control's. And `cut-only` keeps at least 50% of the excess of `all` over the control, although it has only 15% of the pull.

H1 is partial if `whole` removes at least 30% of that excess and `cut-only` keeps at least 30% of it. Then the cut lines carry a real share of the lean, though not all of it, and the rule can still adopt a policy that improves on `all`.

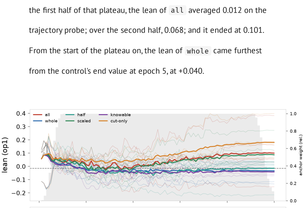
Why: the pool needs only one position of a line to align. On a whole line the op word aligns far more easily than the first operand, so the first operand gets almost no pull; only visits that show nothing else force pull onto it. `half` and `knowable` also drop those visits, so we expect them to remove the lean. `scaled` keeps a sixth of their pull, so we expect it to remove most of it.

Through training, we expect the lean of `all` to build while the anchor weight is high and persist through the anneal, and the lean of `whole` to stay near the control throughout. This prediction is descriptive, with no gate. If `whole` shows a lean early and sheds it later, or `all` builds its lean only late, that would mean the cut lines matter at a particular stage of training, which a reading taken only at the end would miss.

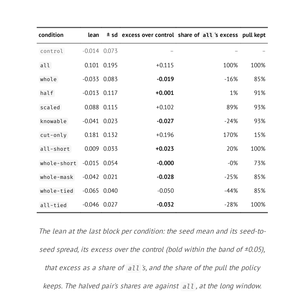
If `whole` keeps most of the lean, the lean comes from whole lines (perhaps through the readout, as the op1-lean reanalysis found for *red*), and cropping is a side issue on this grammar. If `whole` removes part of the lean but stays outside the band, the cut lines are one route among others, and `cut-only` says how large a share they carry. On *red* the smoke test landed here: `whole` removed 29% of the excess, just short of partial. The model arms (H5) test two routes for the rest.

If the lean of `all` is less than 0.1 above the control's, it did not reproduce at these seeds, and H1 is unresolved.

What we saw. `all` leaned 0.101 at the last block, against the control's -0.014: an excess of 0.115, where ex-2.2.14 had 0.20. `whole` removed 116% of that excess, leaving -0.019, inside the band of ±0.05. `cut-only`, with 15% of the pull, kept 170% of it. Of the other policies, `half` removed 99%, `scaled` 11%, and `knowable` 124%.

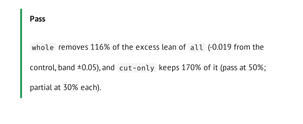


The lean at the last block, per condition. *The mean cosine with e_1 at the first operand, over every op5 probe lines, one small dot per seed and the seed mean as the large mark. The grey band is the control's seed mean ± 0.05, the band `whole` has to land in for H1 to pass; the dotted line is the primary of ex-2.2.14 (0.19). The groups are the policies, the halved windows, and the model arms.*



The lean at each slice. *The seed mean of the cosine with e_1 at the first operand, over every op5 probe lines, from the embedding to the last block, for the control, each policy, and the model arms.*

The anchor weight holds at its peak from epoch 5 to epoch 45. Over the first half of that plateau, the lean of `all` averaged 0.012 on the trajectory probe; over the second half, 0.068; and it ended at 0.101. From the start of the plateau on, the lean of `whole` came furthest from the control's end value at epoch 5, at +0.040.



The lean through training. *Top: the lean at the last block on the trajectory probe lines, recorded every 50 steps; the thick line is the seed mean and the hairlines are the seeds. The grey silhouette is the anchor weight's schedule (right-hand scale, relative to its peak). Bottom: the trailing-fragment lean (H3), pooled over ops. The dashed lines are the control's values at the end of training; the control records no trajectory.*

condition	lean	± sd	excess over control	share of `all`'s excess	pull kept
control	-0.014	0.071	–	–	–
all	0.101	0.195	+0.115	100%	100%
whole	-0.035	0.083	**-0.019**	-16%	85%
half	-0.015	0.117	**+0.001**	1%	91%
scaled	0.088	0.115	+0.102	89%	93%
knowable	-0.041	0.023	**-0.027**	-24%	93%
cut-only	0.181	0.152	+0.196	170%	15%
all-short	0.009	0.053	**+0.023**	20%	100%
whole-short	-0.015	0.054	**-0.000**	-0%	73%
whole-mask	-0.042	0.021	**-0.028**	-25%	85%
whole-tied	-0.065	0.040	-0.050	-44%	85%
all-tied	-0.046	0.027	**-0.032**	-28%	100%

The lean at the last block per condition: the seed mean and its seed-to-seed spread, its excess over the control (bold within the band of ±0.05), that excess as a share of `all`'s, and the share of the pull the policy keeps. The halved pair's shares are against `all`, at the long window.

What we make of it. The cut lines carry the lean: `cut-only`, with a seventh of the pull, leaned further than `all`. Perhaps the whole lines in `all` hold the lean down. On a whole line the op word takes the pull, so the first operand can stay where it is.

`half` and `knowable` remove the lean as `whole` does, since each drops the visits that show the first operand alone. `scaled` keeps most of it, so a sixth of the pull on those visits is enough to teach it.

Take the shares as rough. The lean of `all` ranges from -0.15 to 0.38 over the seeds, and its excess only just clears the readable 0.1. The order of the policies is steadier, and the trailing fragments (H3) show it more clearly.

Over training, the lean of `all` built through the plateau and did not fall back in the anneal. So it accumulates, as the per-step view of cut lines suggested.

> **Pass**
>
> `whole` removes 116% of the excess lean of `all` (-0.019 from the control, band ±0.05), and `cut-only` keeps 170% of it (pass at 50%; partial at 30% each).

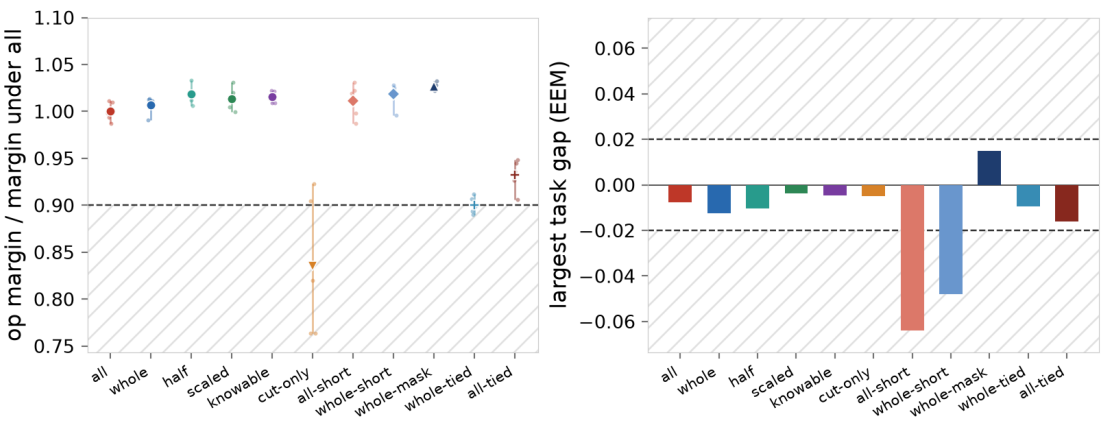
The anchor and the task hold under every policy (H2)

✗ Miss

What we expect. No policy costs the anchor or the task, and neither do the model arms. The op margin of every arm is at least 90% of the margin under `all`, and every op's held-out expected exact match is within 0.02 of the control (the gate from ex-2.2.11), on the seed means.

In ex-2.2.14 the margin sat at the op word and saturated early, and whole lines carry the op word, so dropping cut lines should leave it where it was. If the margin falls under `whole`, there are two possible causes. The cut lines may have been doing part of the work of the anchor. Or the policy removed some of the total pull, and λ_a would need scaling up to make up for it. The pull kept roughly tells the two apart: a fall about the size of the pull removed points to λ_a, and a larger one points to the cut lines. Either way the rule checks this gate, since a policy that moves the anchor or the task would not go forward as it stands.

What we saw. The lowest op margin was 0.84 of the margin under `all`, on `cut-only`. The largest task gap from the control was -0.0639, on `difference` under `all-short`.



The anchor and the task, per arm. *Left: the op margin as a share of the seed-mean margin under `all`, one small dot per seed and the seed mean as the large mark; H2 asks for at least 90% (dashed; hatched below). Right: the gap in held-out expected exact match from the control, for the op furthest from it; H2 asks for at most 0.02 either way (dashed; hatched beyond).*

arm	op margin	share of `all`'s	op of largest gap	gap	pull kept	H2
`all`	0.874	**1.00**	`difference`	**-0.0077**	100%	pass
`whole`	0.880	**1.01**	`difference`	**-0.0126**	85%	pass
`half`	0.890	**1.02**	`difference`	**-0.0104**	91%	pass
`scaled`	0.885	**1.01**	`difference`	**-0.0038**	93%	pass
`knowable`	0.887	**1.02**	`difference`	**-0.0045**	93%	pass
`cut-only`	0.730	0.84	`value-hsv`	**-0.0049**	15%	miss
`all-short`	0.884	**1.01**	`difference`	-0.0639	100%	miss
`whole-short`	0.890	**1.02**	`difference`	-0.0481	73%	miss
`whole-mask`	0.897	**1.03**	`difference`	**+0.0150**	85%	pass
`whole-tied`	0.787	**0.90**	`hsvmix`	**-0.0094**	85%	pass
`all-tied`	0.815	**0.93**	`difference`	**-0.0162**	100%	pass

The two gates of H2 per arm, on the seed means: the op margin and its share of the margin under `all` (bold at or above 90%), and the op whose held-out expected exact match is furthest from the control, with its gap (bold within 0.02).

What we make of it. None of the misses concerns a policy we would take forward. `cut-only` is a diagnostic arm; it removed 85% of the pull and lost only 16% of the margin. Its pull also lands in different places from the others, so it does not measure how the margin depends on total pull by itself. The policies we would take forward keep their margin at the level of `all`, so they need no change to λ_a.

The short pair miss the task gate on `difference` by similar amounts under both policies, which points to the halved window rather than the policy. We have no control at 32 tokens to confirm that.

> **Miss**
>
> 3 of 11 arms miss: `cut-only` (margin 0.84 of `all`'s); `all-short` (task gap -0.0639 on `difference`); `whole-short` (task gap -0.0481 on `difference`). The gates are a margin of at least 0.9 of `all`'s and a task gap within 0.02; over every arm, the lowest margin is 0.84 (`cut-only`) and the largest gap -0.0639 (`all-short` on `difference`).

Trailing fragments without their op word (H3)

✓ Pass

What we expect. Under `all`, trailing fragments lean toward e_1, and under `whole` and `knowable` they lean less. The number is the mean cosine with e_1 over every position of every trailing fragment and every op, at the last block, against the control. This is a prediction of direction, with no gate.

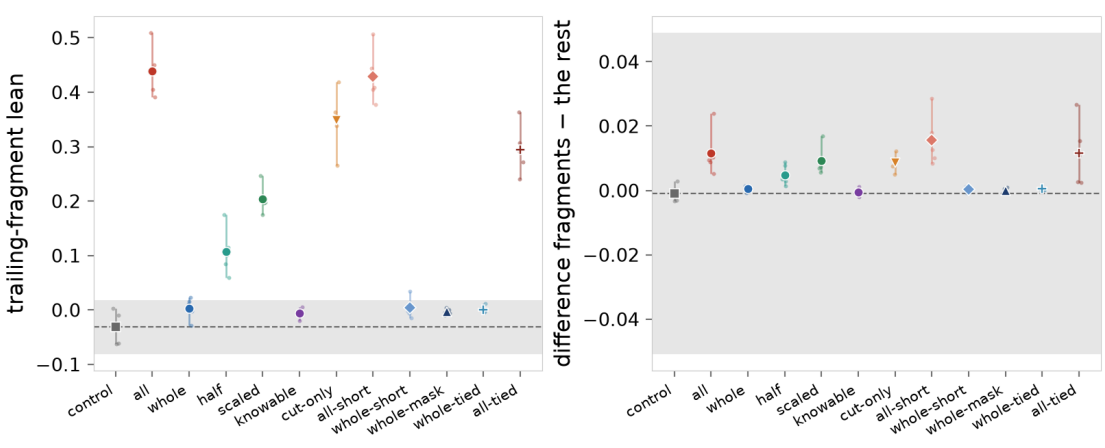
Why: under `all`, the anchor asks a trailing fragment like `op2 = answer ⏎` to align with nothing to go on. The pool puts the pull where alignment comes most easily, which here is likely the syntax tokens `=` and `⏎`, since they carry no color. So we expect most of the lean to sit there.

On *red*, the smoke test points to the second operand as well: on whole probe lines, the lean there came mostly from cut lines. `whole` took it from 0.12 to 0.04, and `cut-only` kept 0.13. But on *red* the second operand can itself be red, so that may not carry over to the op.

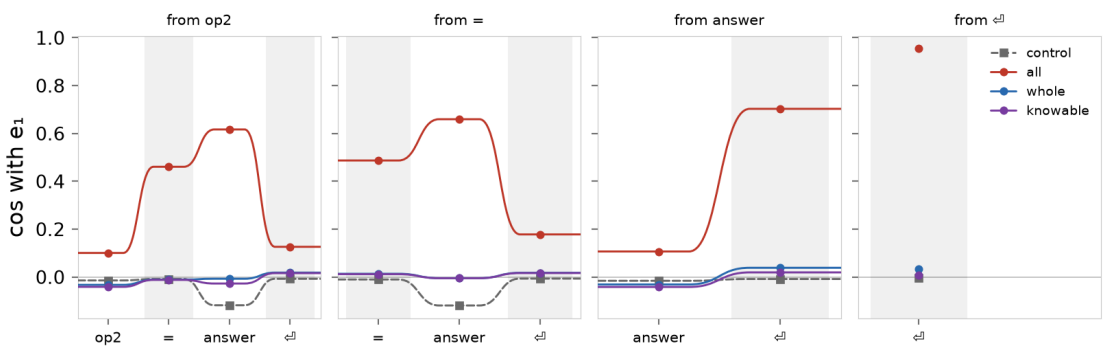
The model can respond with a general lean, as at the first operand, or by guessing the op from what it can see, since some pairs of second operand and answer fit `difference` better than others. So we also report the contrast between the `difference` trailing fragments and the rest, with no prediction.

A positive contrast would mean the model is learning a surface cue for the op: the shortcut the pivot names as a failure mode, here in miniature.

What we saw. Under `all` the trailing fragments leaned 0.439, against -0.031 on the control. Under `whole` they leaned 0.003, and under `knowable` -0.006. The largest lean under `all` sat at `⏎` in the fragments starting at `⏎` (0.954). The contrast between the `difference` fragments and the rest was +0.012 under `all` and -0.001 on the control.



Trailing fragments at the last block. *Left: the mean cosine with e_1 over every position of every trailing fragment of every op. Right: the same over the `difference` fragments less the mean over the rest. One small dot per seed, the seed mean as the large mark; the grey band is the control's seed mean ± 0.05, for scale.*



Where a trailing fragment leans. *The seed-mean cosine with e_1 at the last block at each position of a trailing fragment, pooled over ops, one panel per role the fragment starts at. The shaded columns are the syntax tokens `=` and `⏎`, which carry no color.*

condition	trailing-fragment lean	± sd	`difference` less the rest	± sd
`control`	-0.031	0.030	-0.001	0.003
`all`	0.439	0.046	+0.012	0.007
`whole`	0.003	0.020	+0.000	0.001
`half`	0.107	0.043	+0.005	0.003
`scaled`	0.204	0.026	+0.009	0.004
`knowable`	-0.006	0.010	-0.000	0.001
`cut-only`	0.348	0.055	+0.009	0.003
`all-short`	0.428	0.050	+0.016	0.008
`whole-short`	0.004	0.019	+0.000	0.001
`whole-mask`	-0.003	0.005	-0.000	0.001
`whole-tied`	0.001	0.006	+0.000	0.000
`all-tied`	0.295	0.046	+0.012	0.010

The trailing fragments at the last block per condition: the lean pooled over every op, and the contrast of the `difference` fragments with the rest, each as a seed mean with its seed-to-seed spread.

What we make of it. The shortcut is large: under `all` a trailing fragment leans far more than a first operand does. The lean is strongest on a newline shown alone, where the window leaves nothing else to pull.

The lean is also general. The contrast between the `difference` fragments and the rest is small under every arm, so the model makes little use of what a fragment says about its op.

`half` keeps 29% of the excess fragment lean of `all`, because it still pulls the four-token fragment `op2 = answer ⏎`, which has no op word. `scaled` keeps 50%. Unlike the first-operand lean, the fragment lean varies little across seeds next to the gaps between arms, so it tells the policies apart most clearly.

> **Pass**
>
> The trailing-fragment lean is 0.439 under `all`, against 0.003 under `whole`, -0.006 under `knowable`, and -0.031 on the control.

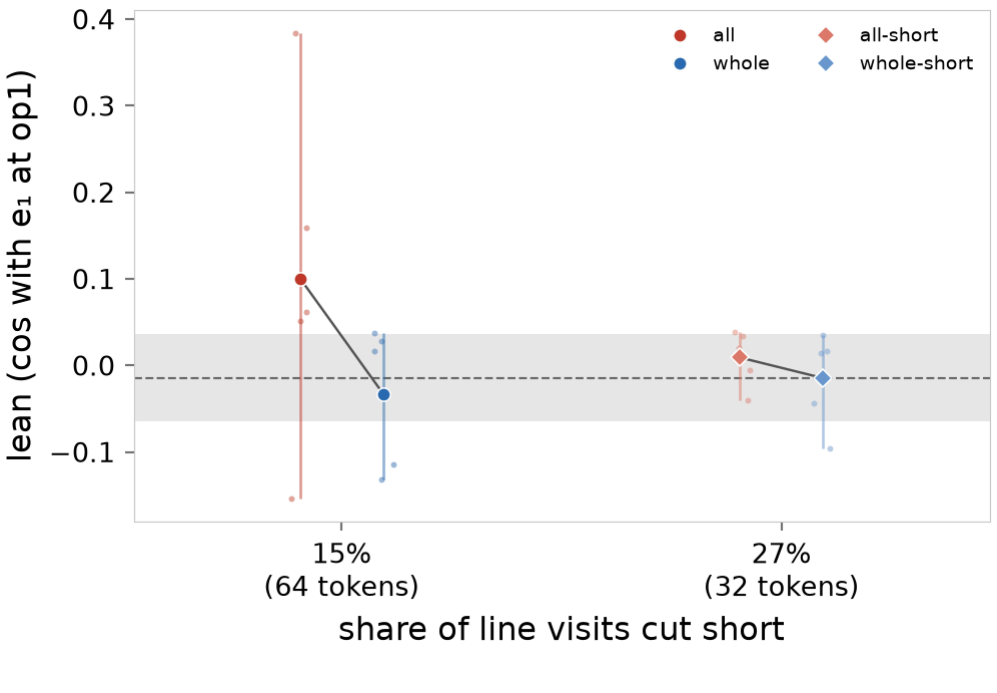
Halved windows, more cut lines (H4) ~ Partial

What we expect. Halved windows give twice as many cut visits, so we expect the lean caused by cut lines to grow. That is, the gap between `all-short` and `whole-short` should be larger than the gap between `all` and `whole`. We also expect `whole-short` to remove the lean, bringing it to within 0.05 of the control. This prediction has no gate of its own; the rule falls back on it if H1 is unresolved.

We compare the gaps within each pair rather than comparing `all-short` with `all`. The doubled batch matches the steps and the anchor updates, but a model trained on 32-token windows has seen less context per line, and the gap within a pair holds that fixed.

Suppose the gap for the halved pair is no larger than the gap for the long pair. Then the lean does not depend on how often the first operand is pulled alone, and the extrapolation to the in-context grammar becomes weaker.

What we saw. At 32 tokens the gap between `all` and `whole` was +0.024, against +0.134 at 64 tokens, so it did not grow. `whole-short` sat -0.000 from the control, inside the band.



The lean against the share of cut line visits. *At each window size, `all` (left) and `whole` (right), one small dot per seed and the seed mean as the large mark, with a line joining the two means: its drop is the lean the cut lines carry. The grey band is the control's seed mean ± 0.05; the control trains at 64 tokens only.*

arm	window	lean	excess over control	gap within the pair	op margin	largest task gap
`all`	64	0.101	+0.115	+0.134	0.874	**-0.0077 (`difference`)**
`whole`	64	-0.033	**-0.019**		0.880	**-0.0126 (`difference`)**
`all-short`	32	0.009	**+0.023**	+0.024	0.884	-0.0639 (`difference`)
`whole-short`	32	-0.015	**-0.000**		0.890	-0.0481 (`difference`)

The two pairs, on the seed means: the lean and its excess over the control (bold within ±0.05), the gap between `all` and `whole` at each window size, the op margin, and the largest task gap from the control (bold within 0.02).

What we make of it. At 32 tokens the trailing fragments lean about as much as at 64 (0.428 under `all-short`, against 0.439 under `all`), so the cut lines teach the shortcut at both window sizes. At the halved window, though, it did not spread to the first operand of whole lines, and we have no explanation for that.

Some of the gap at 64 tokens may be the wide seed spread of `all`, though that would not explain why `all-short` sits so close to the control. Either way, the trailing-fragment lean shows the shortcut at both window sizes, and the first-operand lean does not.

> **Partial**
>
> The gap between `all` and `whole` is +0.024 at 32 tokens against +0.134 at 64, and `whole-short` is -0.000 from the control (band ±0.05).

Where the rest of the lean comes from (H5)

✓ Pass

What we expect. Whatever lean `whole` leaves sits on whole lines, and each model arm removes one route it could take. The number is the seed-mean lean of each arm against that of `whole`, paired by seed. These are predictions of direction, with no gate.

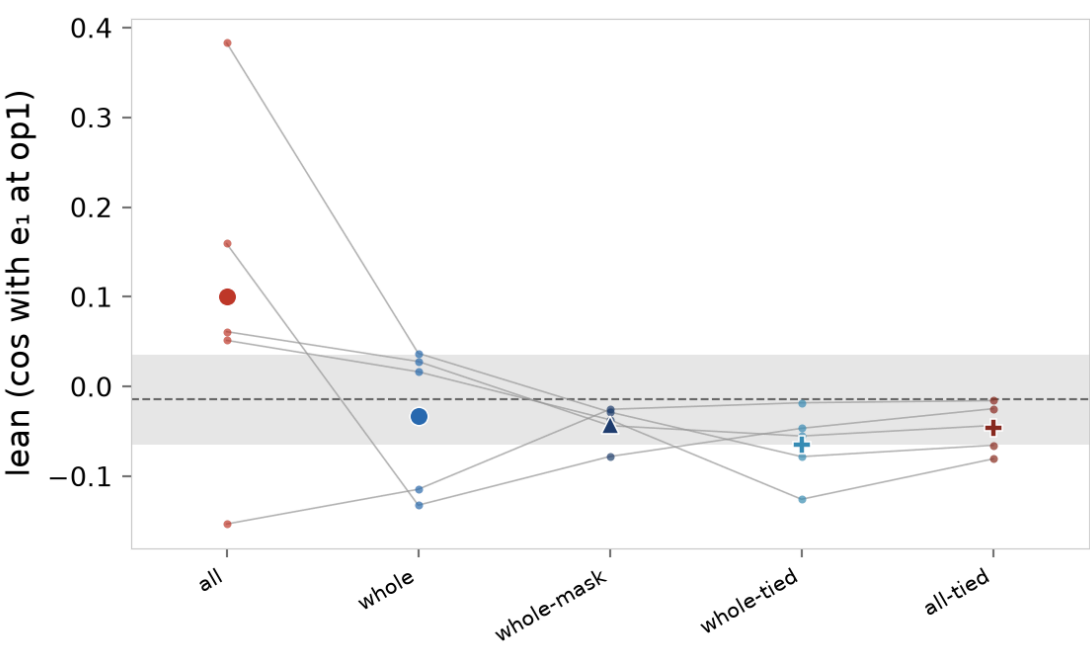
`whole-tied` should lean less than `whole`. The op1-lean reanalysis found that with a readout of its own, the model makes e_1 a feature meaning "a syntax word comes next", and a first operand is always followed by one (the op word). A tied readout shares its rows with the embedding table, and on *red* the reanalysis found that feature almost absent on the tied arm.

`whole-mask` is the weaker expectation. On a whole line the first operand follows a newline and can attend to the line before it. If the lean draws on that line, the mask removes it. If the lean is built at the first operand from its own token, the mask changes nothing. We record the direction either way.

`all-tied` against `whole-tied` asks H1's question under the tied readout, with no gate: whether the cut lines still add lean once the readout is tied. If the tied readout removes the lean from whole lines only, the gap between the tied pair should be about the size of the gap between `all` and `whole`.

If H1 passes, `whole` leaves little lean to split, and these arms mostly say whether each change costs the anchor or the task (H2).

What we saw. Paired by seed, `whole-tied` leaned -0.032 against `whole` (lower at 3 of 5 seeds), and `whole-mask` -0.009 (lower at 3 of 5). Under the tied readout, `all` leaned +0.019 above `whole`, against +0.134 under the untied readout.



The lean under the model arms, by seed. *The lean at the last block, one dot per seed and the seed mean as the large mark. The thin grey lines join the dots of one seed, which trains on the same windows and labels in every column. The grey band is the control's seed mean ± 0.05.*

arm	lean	less `whole`'s, paired	± sd	op margin	largest task gap
`whole`	-0.033	–	–	0.880	**-0.0126 (`difference`)**
`whole-mask`	-0.042	-0.009	0.075	0.897	**+0.0150 (`difference`)**
`whole-tied`	-0.065	-0.032	0.114	0.787	**-0.0094 (`hsvmix`)**
`all-tied`	-0.046	-0.013	0.107	0.815	**-0.0162 (`difference`)**

The model arms against `whole`: the seed-mean lean, its difference from `whole` paired by seed (with the spread of that difference over seeds), the op margin, and the largest task gap from the control (bold within 0.02).

What we make of it. `whole` leaves no lean to explain, so the pass of the two model arms says little. The informative arm is `all-tied`.

With the readout tied, the cut lines no longer make the first operand of whole lines lean. But the trailing fragments still lean (0.295), and the ⏎ embedding keeps its component on e_1 (+0.160 under `all-tied`, +0.162 under `all`, against -0.003 under `whole`). So the tied readout closes the route from cut lines to whole lines that the op1-lean reanalysis found, and leaves the shortcut on the fragments in place. A crop policy that drops the trailing fragments, as `whole` and `knowable` do, handles both.

> **Pass**
>
> `whole-tied` leans -0.032 against `whole`, paired by seed, and `whole-mask` -0.009.

The rule for the pilot

> If H1 passes or is partial, the pilot's default crop policy is `scaled` if it removes at least half the share of the excess lean (H1) that `whole` removes, and it passes H2. Otherwise it is whichever of `half` and `whole` keeps more of the pull while it removes as much of the lean as H1 asks of `whole` (to within the band of the control if H1 passed, at least the partial share if it was partial) and passes H2. If H1 missed, or no policy qualifies, the pilot keeps `all`, and leans on label variant (c) for the evidence question. If H1 is unresolved, the choice falls to the short pair: `whole` goes forward if the lean of `all-short` exceeds the control's by the readable margin, the lean of `whole-short` is within the band of it, and `whole` passes H2; `all` stays otherwise.

The test for `scaled` is looser than for the others, and since it is stated against `whole`, it loosens with H1. `whole` may give the cleanest result, but only `scaled` extends to labelled spans of any length, so the rule accepts part of the lean to get it. The model arms do not enter the rule.

With H1 at *pass*, the rule goes forward with **`half`**.

policy	share of the excess removed	the lean test	H2	pull kept	qualifies
`scaled`	11%	at least 58%: no	pass	93%	no
`half`	99%	within ±0.05: yes	pass	91%	yes
`whole`	116%	within ±0.05: yes	pass	85%	yes

*The candidates against the rule, in the order it considers them, with H1 at pass. The share removed is of the excess lean of `all` over the control; `scaled`'s bar is 50% of what `whole` removes. The rule goes forward with **`half`** (bold).*

`half` keeps 91% of the pull. `scaled` missed its test by a wide margin, so no qualifying policy pulls spans of every length: `whole` never pulls a span longer than the window, and `half` stops at twice it.

The pilot starts from `whole` instead, a choice made after seeing the results. `half` still pulls the four-token trailing fragment, so it keeps 29% of the excess fragment lean (H3). Its cutoff at half the tokens also has no tie to where the evidence sits: in the in-context grammar, a context more than half in view may still have lost the examples that name the op.

The reason for `whole` is plainer: a whole line always shows its evidence. It gives up 15% of the pull.

Exploratory analyses

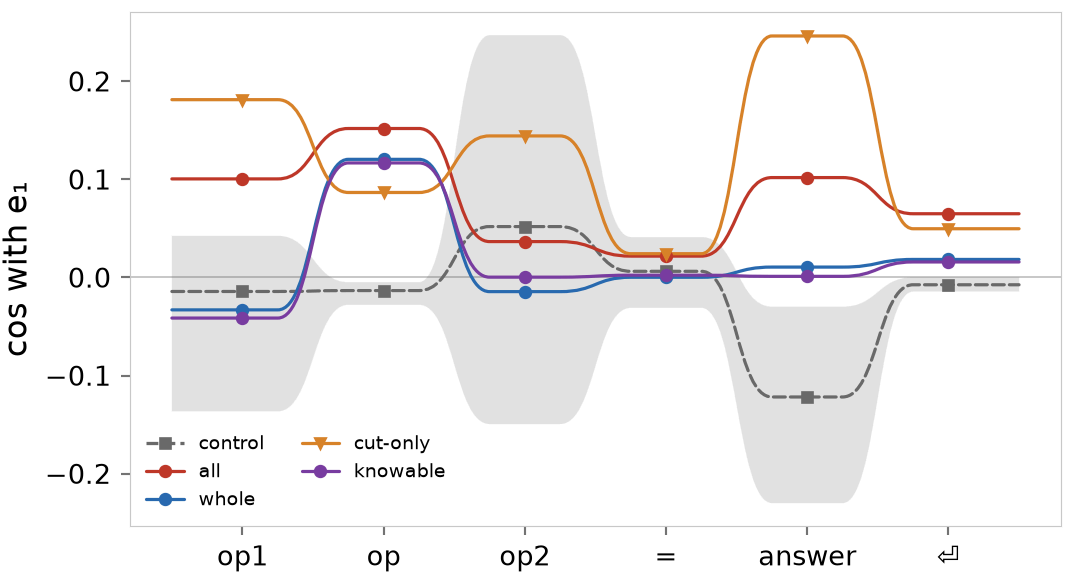
Anything we think of after seeing the data goes here, marked as post hoc. Three descriptive measurements are planned, with no gate.

Where the pull lands. For each arm and slice, the share of the pull on a labelled line that goes to each role, averaged over the ways a window shows a line (as in E7 of op1-lean). This shows what each policy changes about where the anchor is asked to act.



Where the pull lands (post hoc; planned, no gate). *The share of the pull on a labelled* `difference` *line that goes to each role at the last block, averaged over the ways a window shows the line, per policy. A row sums to the pull the policy keeps, relative to a line under* `all`*.*

Each role on whole lines. The mean cosine with e_1 at each role over every op's probe lines, per arm and slice. A line cut before its op word puts its pull on the roles after it, so those roles may lean the way the first operand does, on whole lines as well as on trailing fragments.



Each role on whole lines (post hoc; planned, no gate). *The seed-mean cosine with e_1 at the last block at each role, over every op's probe lines, which are whole lines. The grey band is the range of the control's single seeds at each role; a mean over five seeds varies less than that.*

The tied readout. On the tied pair against `all` and `whole`, the measurements the untied readout made hard in earlier runs: the readout gap from the op1-lean reanalysis (how much of the log-odds of a syntax word against a color at the first operand rides on e_1), the component on e_1 of the ⏎ embedding row, and whether the op margin drifts down before the anneal, which ex-2.2.10 saw only with the untied readout.

condition	syntax-word log-odds at op1	of it on e_1	⏎ embedding on e_1	margin drift before anneal
`control`	8.63	+0.11	-0.034	–
`all`	8.72	-0.59	+0.162	-0.033
`whole`	8.61	+0.18	-0.003	-0.015
`whole-tied`	9.04	-0.13	-0.013	-0.031
`all-tied`	9.14	-0.12	+0.160	-0.017

The tied readout against the untied (post hoc; planned, no gate), seed means. The log-odds of a syntax word against a color at the first operand, and the part of them that rides on e_1 (the drop when e_1 is removed from the state, as in the op1-lean reanalysis); the e_1 component of the ⏎ embedding row; and the op margin at the start of the anneal less its peak before it, on the trajectory.

The ⏎ embedding row is where the fragment lean shows up in the weights: on a visit that shows only the newline, that row is the one parameter specific to the position being pulled.

Discussion

The pilot starts from `whole`, with the untied readout from the handover recipe. The tied readout closed only the route from cut lines to whole lines (H5), so the crop policy does the work either way. The tied arms passed H2, though with the lowest margins of any arm except `cut-only`, so the technique works with the tied readout too, at some cost in margin.

Halving the window here doubled the share of cut visits and left the fragment lean about where it was (H4), so a larger share of cut contexts may not matter much by itself. The trailing-fragment lean is the measurement to watch in the pilot, alongside the first-operand lean: it measures the shortcut itself and varies little across seeds. Here, `whole` held it at the level of the control.

`knowable`, our version of label variant (c), removed both leans too. It needs to know where the evidence is, so it can't be adopted, but it is worth keeping in the pilot as an oracle, since it bounds what a window-only policy could reach.

Two questions stay open: what policy suits spans longer than the window, given that `scaled` taught the shortcut with as little as a sixth of the pull, and why halving the window shrank the first-operand lean while the fragments leaned as before (H4). The newline mask added nothing measurable on top of `whole`, though `whole` left no lean for it to remove.

Method

The policies

The sampler already knows the visible run of each labelled line: the offset of the window fixes the role of every position, and padding hides a prefix. Each policy turns that run into a weight in [0, 1] on the pooled term of the line (`line_weight` in `experiment.py`).

The denominator stays the count of labelled lines with any visible position, as under `all`, and the label draws are unchanged, so the random stream is consumed identically under every policy. `knowable` also removes the positions before the op word from the pool, so a whole line is pooled over roles 1 to 5.

How often a window shows each run of a line, at each window size, enumerated over every offset and padding length:

visible roles	what the anchor can see	64-token window	32-token window
op1	**no op word**	1.5%	2.7%
op1 op	the op word, cut short	1.5%	2.7%
op1 op op2	the op word, cut short	1.5%	2.7%
op1 op op2 =	the op word, cut short	1.5%	2.7%
op1 op op2 = answer	the op word, cut short	1.5%	2.7%
op1 op op2 = answer ⏎	the whole line	85.3%	72.6%
op op2 = answer ⏎	the op word, cut short	1.5%	2.7%
op2 = answer ⏎	**no op word**	1.5%	2.7%
= answer ⏎	**no op word**	1.5%	2.7%
answer ⏎	**no op word**	1.5%	2.7%
⏎	**no op word**	1.5%	2.7%

The measurements

The lean is the mean cosine with e_1 at role 0 over the probe lines of every op, at slice 4, as ex-2.2.14 measured it. Its excess is measured against the seed mean of the control. The op margin is the one from ex-2.2.14, on the same probe sets. The task is held-out expected exact match per op, against the control from ex-2.2.11.

`whole-mask` is measured with its mask on, since the mask is part of the model; on a probe fed one line at a time it changes nothing.

The trailing fragments are the probe lines of every op, cut to start at roles 2, 3, 4, 5 and fed as sequences of their own. The trailing-fragment lean is the mean cosine over every position of every trailing fragment. The contrast is the mean over `difference` trailing fragments less the mean over the rest.

The op margin, the lean, and the trailing-fragment lean are also measured during training, every 50 steps, on the same probe lines. These measurements run inside the training loop; no checkpoint is kept along the way.

Budget

55 runs at d64-L4, each as long as a run in ex-2.2.14; the halved pair takes the same steps at twice the windows per step. Ex-2.2.13 trained 160 runs of this size for about fourteen dollars on Modal. Eval adds the trailing-fragment probe to the measurements from ex-2.2.14; no intervention is scored.

What this experiment does not do

It does not change what the model sees. The plan in the pivot for the in-context grammar keeps random crops of the packed corpus, with a block large enough to fit at least two whole contexts, so the first context in a window is usually cut short. Cut lines could be removed altogether with windows that hold whole lines only: each window starts at a line boundary and is padded after the last line that fits, with a longer block so the padding is a small share. Windows that start at a line boundary but are not padded would still cut the last line. Either changes the task data as well as the pull, so it would need a control of its own. A policy from this experiment still matters with padded windows wherever a labelled span can be longer than the window, as a natural-language document can. The newline mask leaves cut lines cut, so `whole-mask` tests it beside a crop policy, and does not replace one.

It also does not test label variant (c) on the in-context grammar; `knowable` is its version on this grammar.